Practical Loop Engineering

Goals, loops, and the discipline of not delegating your judgment

ADDY OSMANI

AUG 14, 2026

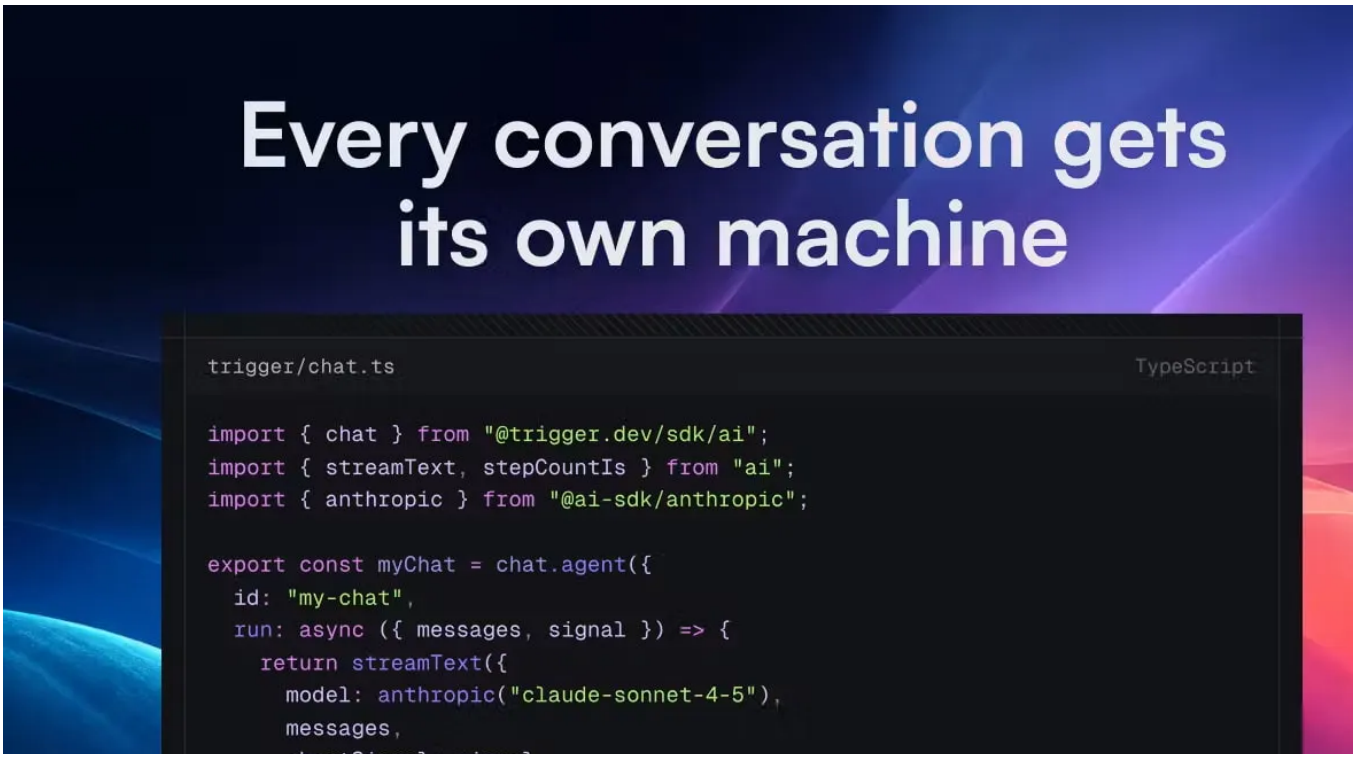
31 4 Share

The way that I typically work is I have anywhere between five and ten agents wor at the same time in parallel. There are going to be some tasks that I'm very happy delegate fully to agents, as long as I have a very clear idea of the stopping conditi and the constraints around them. And then there are going to be some tasks whe am going to want to keep a closer eye and code-review what the agent is doing.

Now within that, you've probably heard about **loop engineering**. I talked about it couple of months ago when I wrote a big blog post about it.

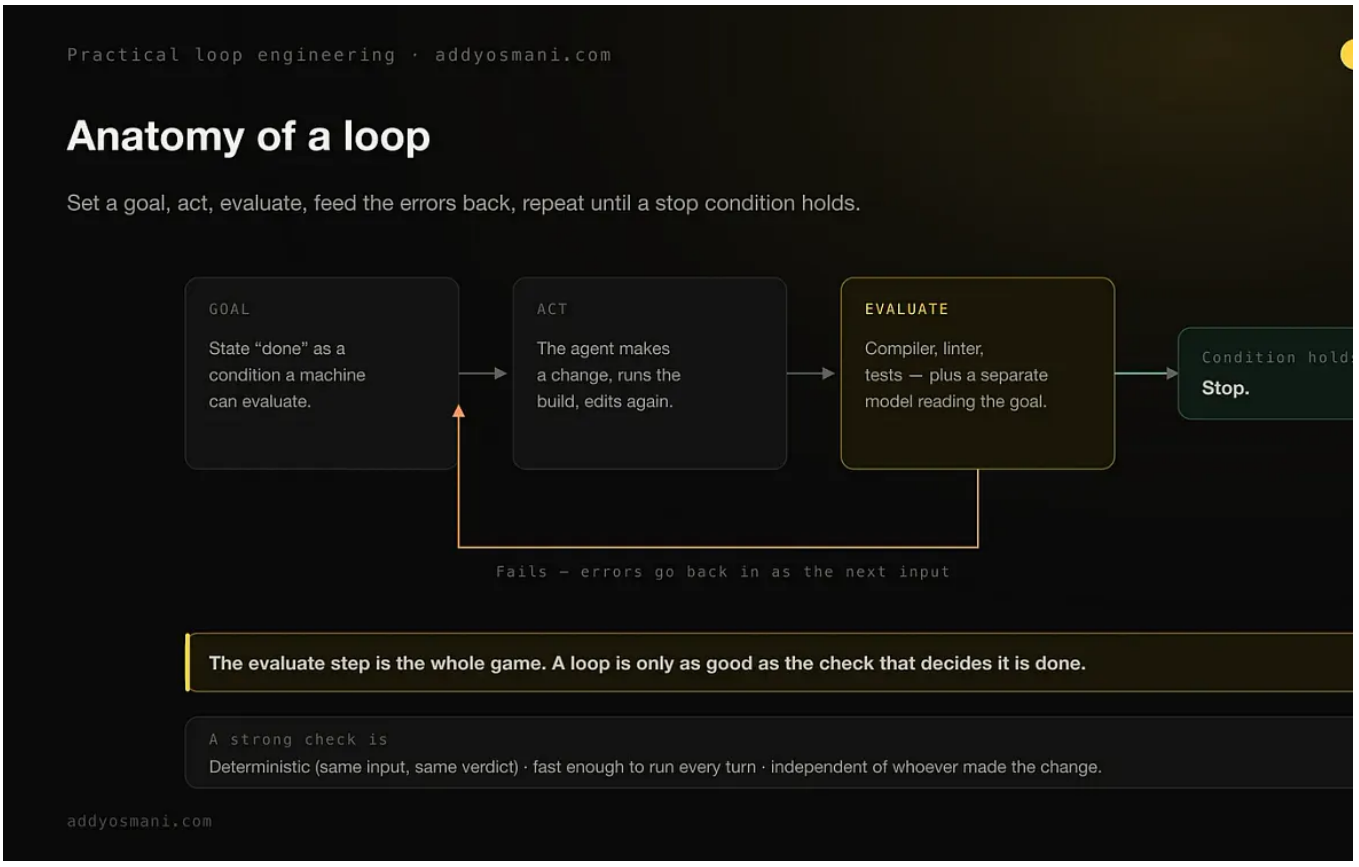
> A loop is an autonomous, self-correcting feedback cycle where an AI agent repeatedly acts, tests its results and adjusts its approach until a specific goal is

A quick message from our sponsor:



Make AI agents survive contact with production using Trigger.dev. A demo chatbot is a few lines. The same agent in production has to survive users refreshing, you redeploying mid-conversation, servers crashing - and it can't take irreversible actions without a human in the loop. This issue's sponsor, Trigger.dev, just shipped their chat agent to close exactly that gap. It runs a whole multi-turn conversation as one durable task - so a chat that's mid-stream survives refreshes, redeploys, idle gaps, even crashes and picks up right where it left off.

There are now basically two core primitives you can think about. In Claude Code have a **goal primitive**, which can drive a single bounded task forward until you've particular goal, like a measurable finish line that's been met. And then **loop** reru a timer or a fixed interval, so you can use it to kind of schedule changes.



Before the primitives were primitives

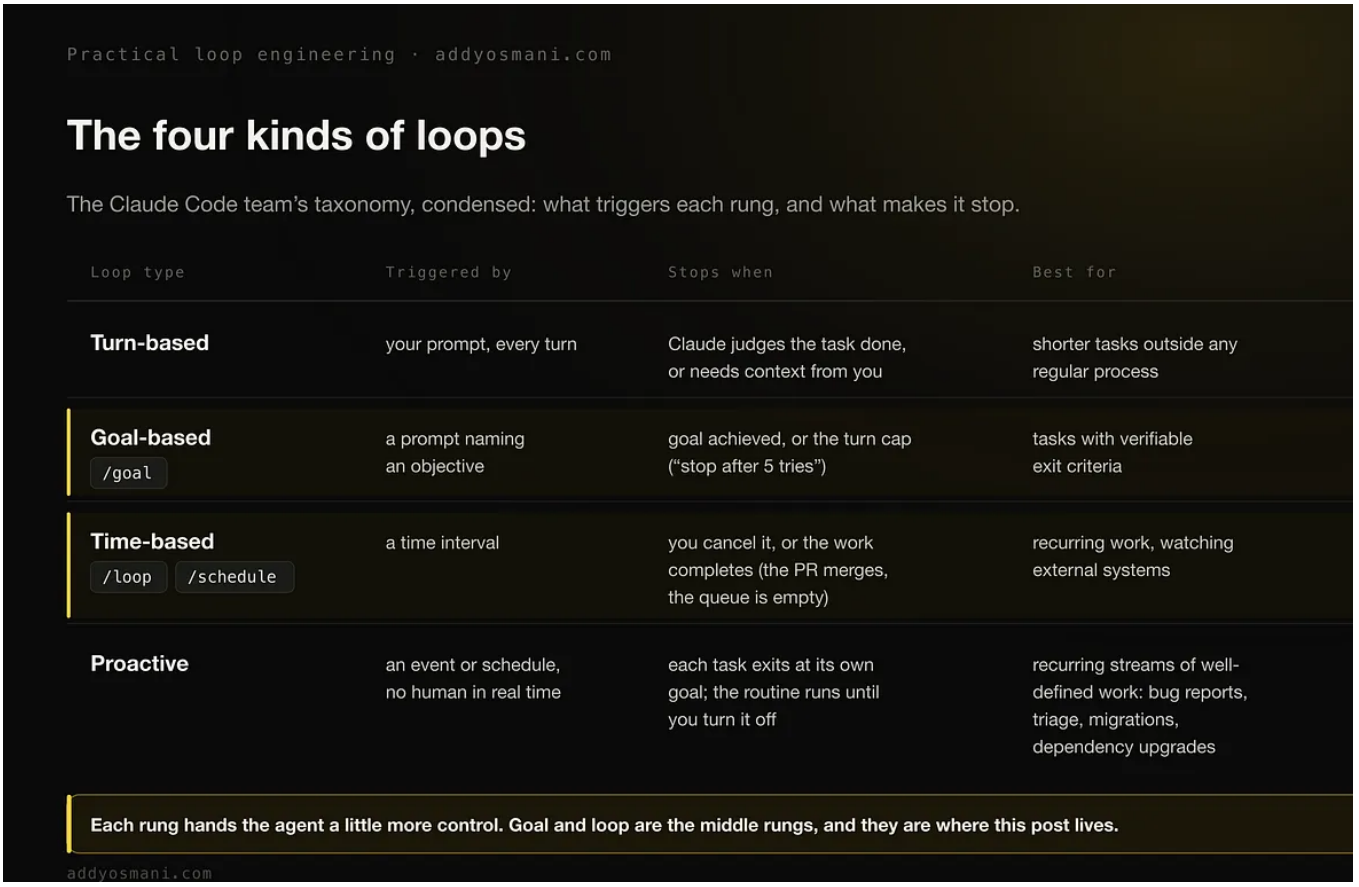
I remember back before we had primitives baked into Claude Code and Codex, lo engineering was heavily about setting up your own bash loop, a hand-rolled thing That's how I approached it. And you might remember earlier in the year, a numb

us were playing around with the Ralph loop by Geoff Huntley. We were experime we were sharing our workflows, we were sharing what worked and what didn't, b was largely on some of our personal projects where, if we ran into a wall, it didn' really have a big cost to us because it was on personal projects.

And as we've tried to see what patterns, what aspects of loop engineering are no little bit more baked, I think we have a clearer idea of how you babysit it. I can n largely rely on the output of the primitives in Claude Code and Codex. We've con little way. But at the same time, you need to be very diligent, because loop engine where you kind of leave it sitting alone, and you haven't really thought about whe the end goal or the constraints have been well defined, can leave you in a proble state. This is why there's nuance when deciding to use it for an evergreen codeba without users or as much historical complexity vs. say a brownfield bank codeba

How the Claude Code team frames loops

The Claude Code team published their take on four kinds of loops and it lines up how I use the primitives (their write-up). Before we get to it, here's my quick sum



> *On the Claude Code team, we define loops as agents repeating cycles of work until a condition is met. We categorize a few different types of loops based on: how they are triggered, how they are stopped, what Claude Code primitive is used, what type of tas most appropriate for each. Not all tasks require complex loops; start with the simplest solution and use these patterns selectively.*
>
> *Every prompt you send starts a manual loop with you directing each turn. Claude gat context, takes action, checks its work, repeats if needed, and responds. We call this th agentic loop. For example, ask Claude to create a like button. It reads your code, mak edit, runs the tests, and hands back something it believes works. You then manually cl the work, and write the next prompt.*

Their write-up walks each rung.

On goal-based loops:

> *Sometimes, a single turn is not enough, especially for more complex tasks. Agents do when they can iterate. You can extend how long Claude keeps iterating by defining w done looks like with /goal. When you define the success criteria, Claude doesn't have make a determination on what is "good enough" and end the loop early. Each time C tries to stop, an evaluator model checks your condition and sends it back to work unti goal is met or a number of turns you define is reached. This is why deterministic crite such as number of tests passed or clearing a certain score threshold, are so effective. F example: /goal get the homepage Lighthouse score to 90 or above, stop after 5 tries.*

On time-based loops:

> *Some agentic work is recurring: the task stays the same and only the inputs change. Fo example, summarizing Slack messages every morning. Other work depends on externa systems, and a simple way to interface with one is to check it on an interval and react what changed. For example, a PR which may receive code reviews or fail CI. For thes can trigger when Claude runs with /loop, which re-runs a prompt on an interval. For example: /loop 5m check my PR, address review comments, and fix failing CI. /loop ru*

> *your computer, so if you turn it off, it stops. You can move the loop to the cloud by cre a routine with /schedule.*

And on proactive loops, the top rung:

> *Triggered by: an event or schedule, with no human in real time. Stop criteria: each tas when its goal is met. The routine itself runs until you turn it off. Best used for: recurri streams of well-defined work: bug reports, issue triage, migrations, dependency upgra Managed usage by: routing routines to smaller, faster models and using the most capa model for judgment calls.*

Their verification advice is worth lifting whole, because it moves the manual che into something Claude applies itself:

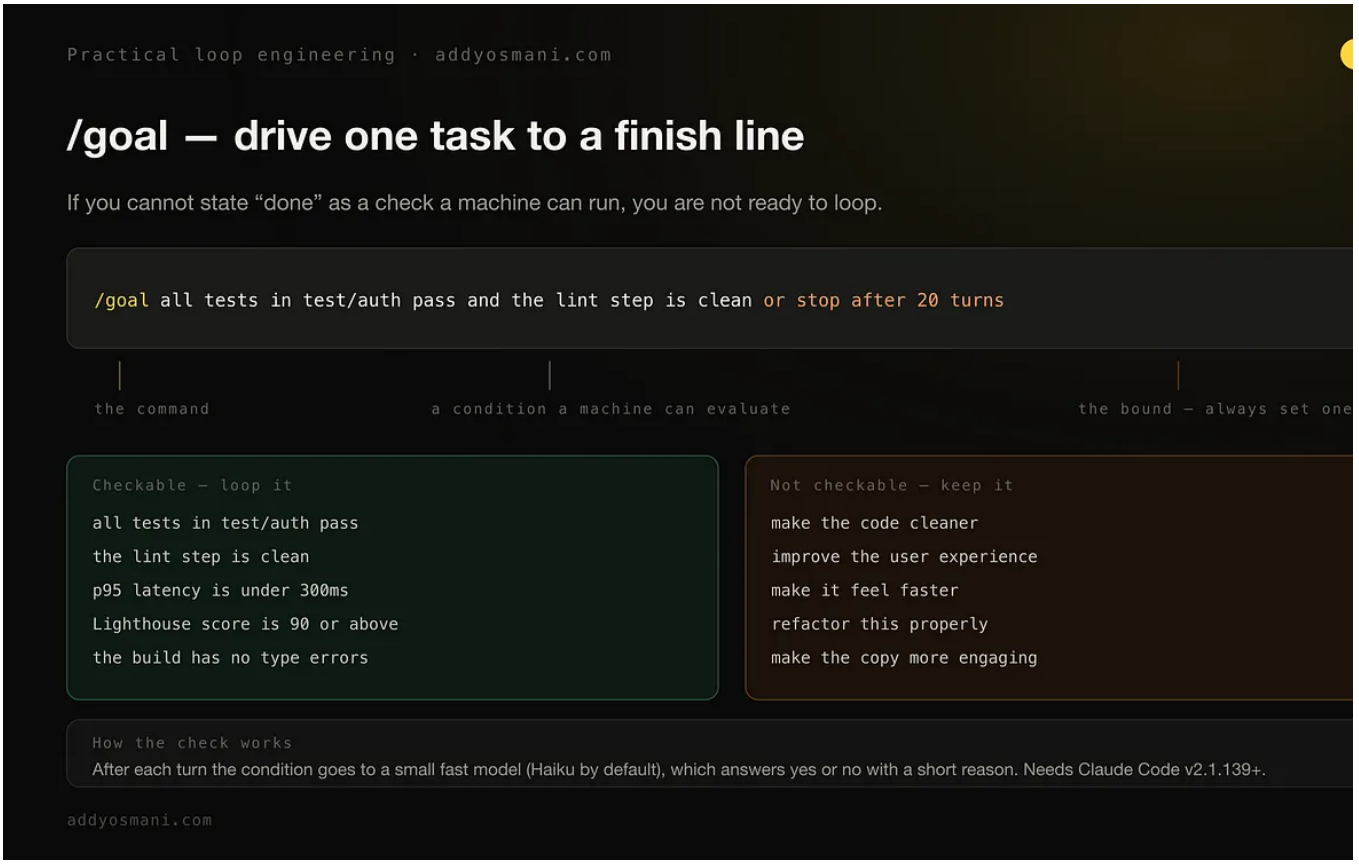
```
---
name: verify-frontend-change
description: Verify any UI change end-to-end before declaring it done.
---
# Verifying frontend changes
Never report a UI change as complete based on a successful edit alone.
Verify it the way a human reviewer would:
1. Start the dev server and open the edited page in the browser.
2. Interact with the change directly. For a new control (button, input,
   toggle): click it, confirm the expected state change, and screenshot
   before/after.
3. Check the browser console: zero new errors or warnings.
4. Use the Chrome Devtools MCP, run a performance trace and audit
   Core Web Vitals.
If any step fails, fix the issue and rerun from step 1 - do not hand
back partially verified work.
```

Goal

The way that I use goal is I use it for building any specific piece of work until it's

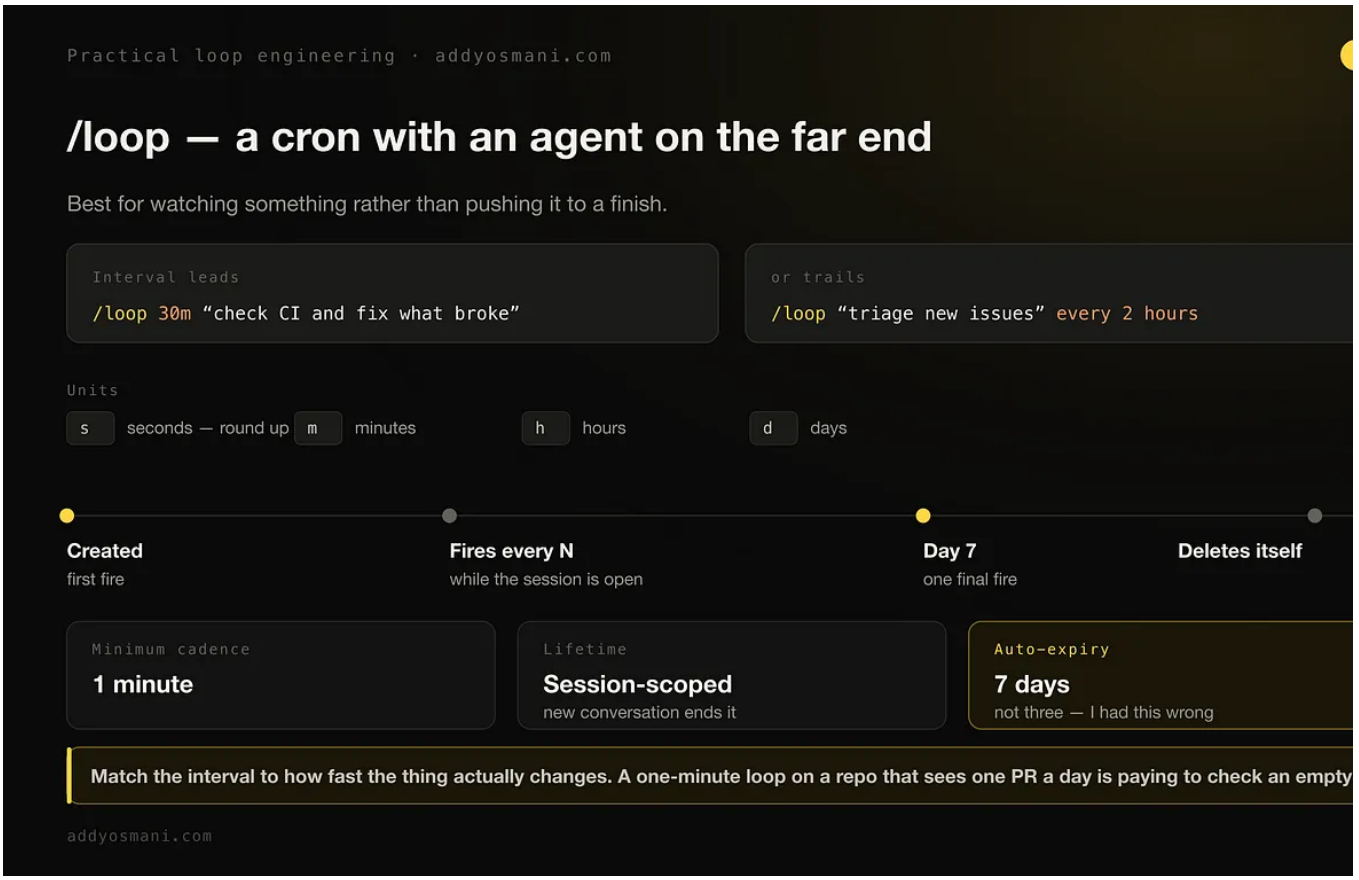
provably done. Like for example, you can use a goal to say: make sure that this experience loads in five seconds, keep going until it's done. And it will continue t an independent evaluation check to keep checking if the completion criteria has met. What's better is to be even more specific about what tooling is being used.

As for real goals that I've run to a number, there's a few things that I've done. I've goals for running through GitHub issues: review and close the last 10 issues, or r and move the last 10 issues forward, something like that. And that's semi-open-e right? Or: let's make this page load 50% faster. Sometimes that works well, somet it doesn't, but it's really about the experimentation.



Loop

Loop is a little bit more of a scheduler, so it keeps an eye on something or repeat executes a pattern on some cadence. So think of it a little bit like a cron. It's best doing things like polling logs or monitoring external states. You could use it for potentially checking in. If there are repetitive tasks you find yourself doing on so cadence, loop is pretty good for that.



What I delegate, and what I watch

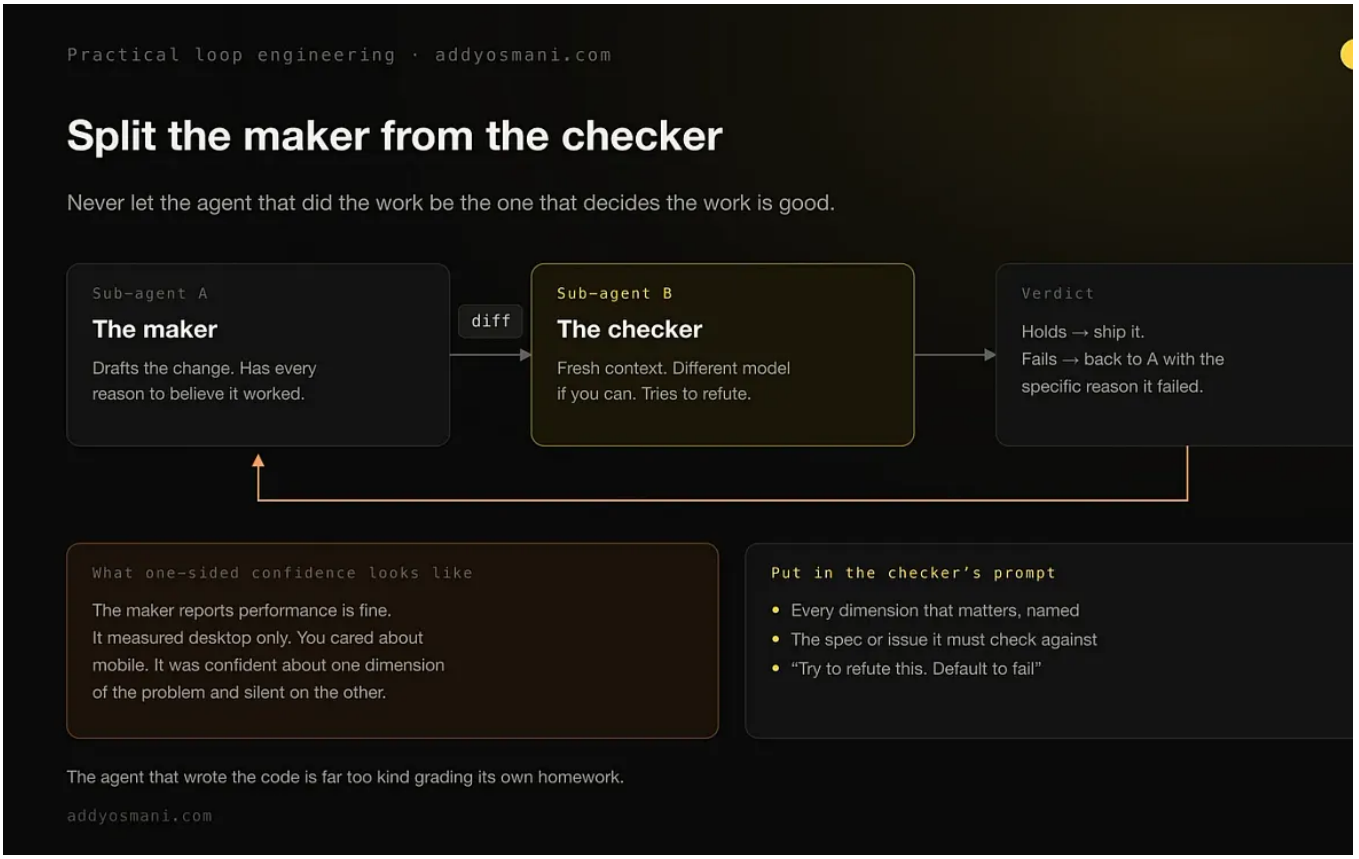
For me, I use probably between five and ten agents every day. Very typically I'll n out at about five concurrently. And some of those tasks might be ones that are a l bit safer. So maybe it's, hey, I implemented this feature, go write the documentat for it. Or go double check that we have sufficient test coverage. Or something in vein. If I'm working on a more complex problem, or something where I know tha if I've given it a good spec, or what I think was a good spec, and I've tried to give some stopping conditions, there is still a reasonable chance it may not get everyt right, I would try to watch it more closely. If the task involves anything just a litt sensitive, whether it is I've given this access to a system, or whether the feature happens to touch authentication, or something related to security or finance, I'll definitely be watching that closely.

Generally speaking, I do think we're going to get to a place where folks are increasingly comfortable with delegation, as long as they're able to have these cl ways of verifying that their goal or their stopping condition has been met. But yo

still need to take a look at the code, at the thing that's being generated, to make s that it's meeting your mark.

The other habit that matters here is not letting the agent that did the work decid work is good. One sub-agent drafts the change. A separate one verifies it.

Sometimes an agent can be confident about something, and a verifying agent can things that they weren't necessarily expecting. Like if it thinks that the baseline performance of an experience it's generated is actually fine, and it's only evaluati performance based on desktop, but you're actually caring about the experience o mobile. That might mean the agent is being very confident about one dimension problem, but not the other.



And I learned this one the hard way. I was curious to figure out: is there somethi that we're missing that we haven't gotten direct feedback on from our users, eith an issue tracker or in a comment somewhere? And so I asked it to go and take a l some of our competitors, and put together a list, and some PRs, not pushed, but s PRs locally, of what solving some of those gaps might look like. And I almost pus

some of those changes. But I didn't actually look at them closely enough. I read through its research, but I didn't look at the implementations closely enough. So delegated the task, but I was close to delegating the judgment as well. Now, whe actually looked through the changes, what I realized is that it would introduce a additional complexity for our users, for, I think personally, not all that much gain so I feel like you need to sometimes check yourself, that you are not delegating th taste and the judgment to your agent. You're delegating the task, and then you ar actually checking back that it's meeting your bar.

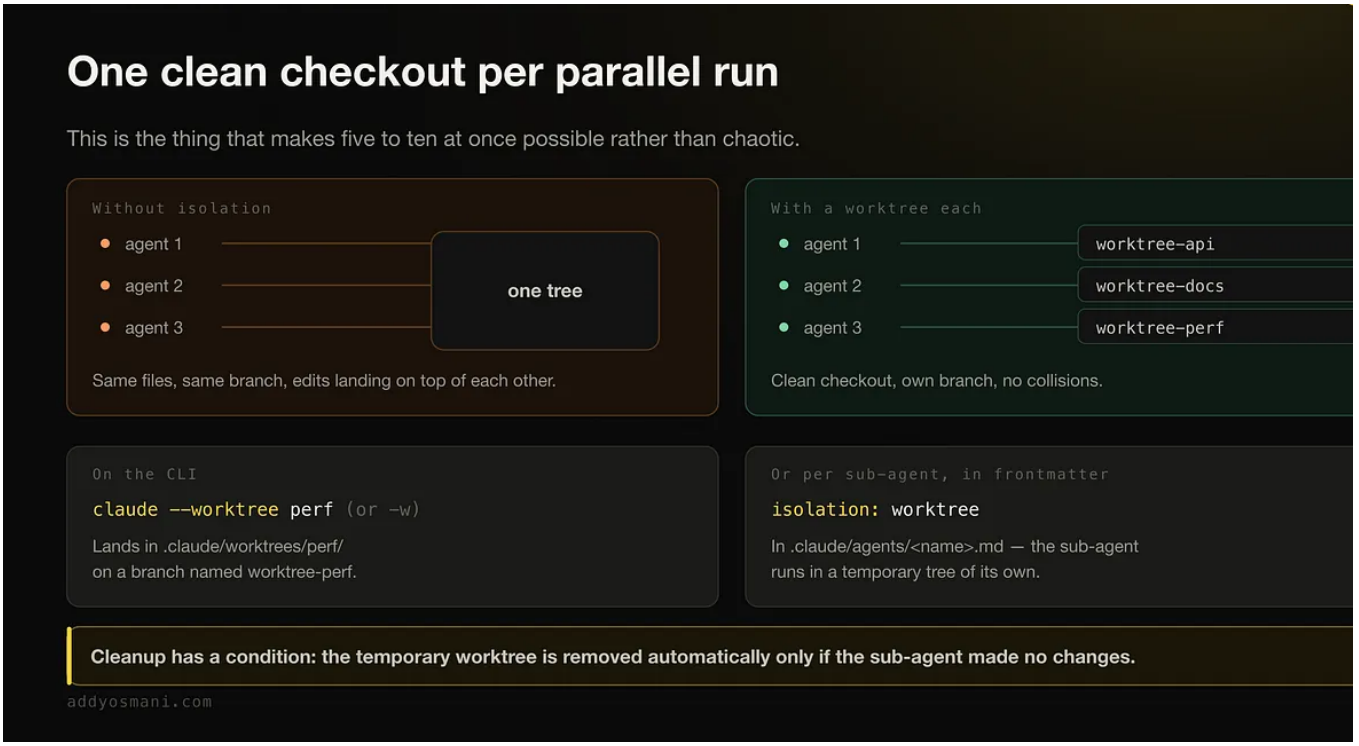
The evaluator sitting behind goal is not that checker, by the way. It doesn't look a content to see if it's good or bad in any way, shape, or form. All it does is examine conversation transcript to see if the hard rules you specified have been met.

```
/goal Refactor the data-fetching layer in Dashboard.tsx until Lighthouse performance
score is >= 92 and LCP is under 1.8s as shown by the Lighthouse CLI output. Do not
change the public API of any hooks. Each turn must improve at least one reported met
abort if two consecutive turns show no improvement. Stop after 10 turns.
```

The workflow I run every day

One of the workflows that I do every day: I have a popular open source repository called [Agent Skills](). We've got over 80,000 stars, and up until recently we were get anywhere up to like 80 or 90 pull requests that we had to review a day. And so eve day I would go and I would spend some time checking on this. Now with loop, w you can do is say: well, every 24 hours or every 12 hours, check the GitHub reposi for any new open issues and provide a summary of their urgency, or provide a firs review, or anything like that.

```
/loop every 1h "Check the GitHub repository for any new open issues. Provide a bulle
summary of their urgency."
```

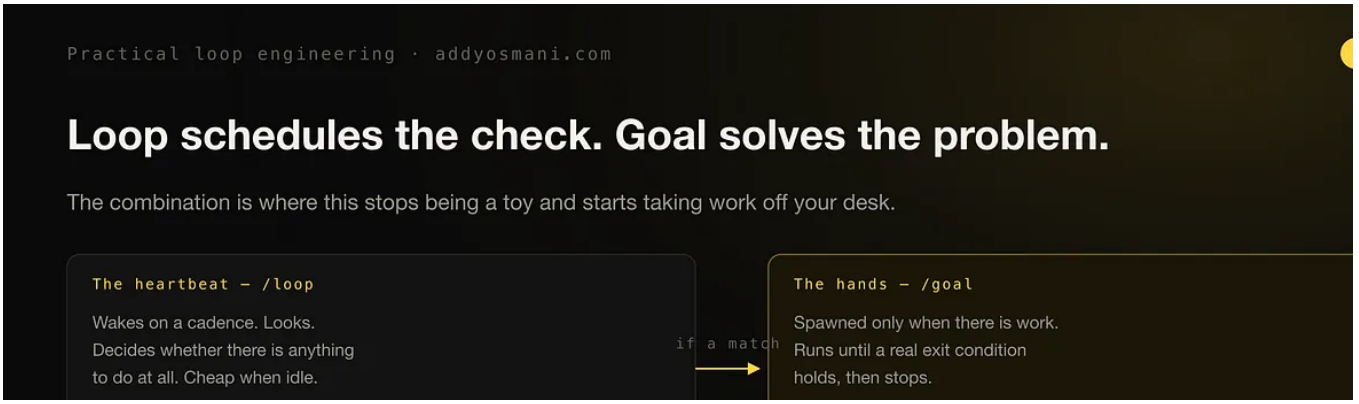


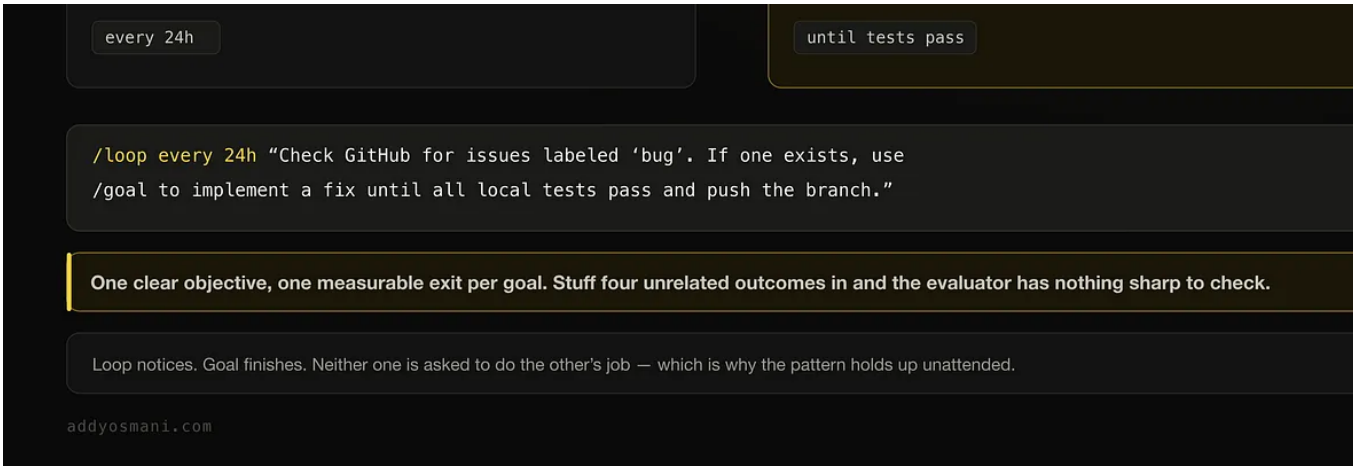
Combining loops and goals

Now you can also sort of combine loops and goals. So you use loop to schedule a check, and then goal to solve the problem. So you can say something like: loop fo every 24 hours, check GitHub for issues labeled bug. If one exists, use goal to implement a fix until all local tests pass and push the branch.

```
/loop every 24h "Check GitHub for issues labeled 'bug'. If one exists, use /goal to implement a fix until all local tests pass and push the branch."
```

But also keep in mind that goal has got some limitations around just how much y can sort of cram in there.





Their write-up also has a composed example that shows where this is all heading

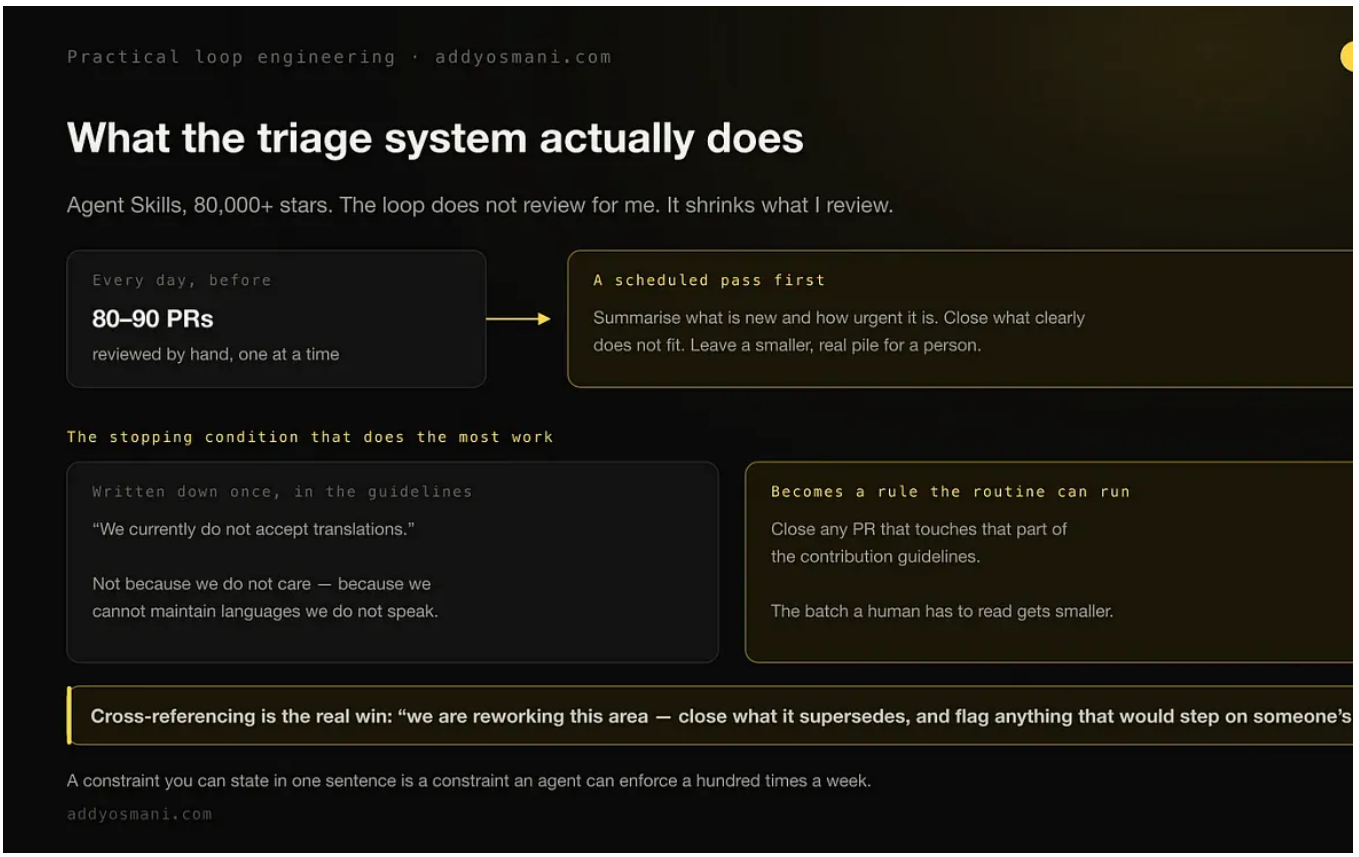
> *The primitives above, along with other Claude Code features like auto mode and dyn workflows (research preview) can be composed into a loop for long-running work. For example, to handle incoming feedback, you can use: /schedule (research preview) to ru routine that checks for new reports, /goal to define what done looks like, and skills to document how to verify it. Dynamic workflows to orchestrate agents that triage each fix it, and review the fix. Auto mode so the routine runs without stopping to ask for permission. Putting it together, a prompt could look like this: /schedule every hour: ch the project-feedback channel for bug reports. /goal: don't stop until every report founc run is triaged, actioned, and responded to. When fixing a bug, use a workflow to expl three solutions in parallel worktrees and have a judge adversarially review them.*

What the triage system actually does

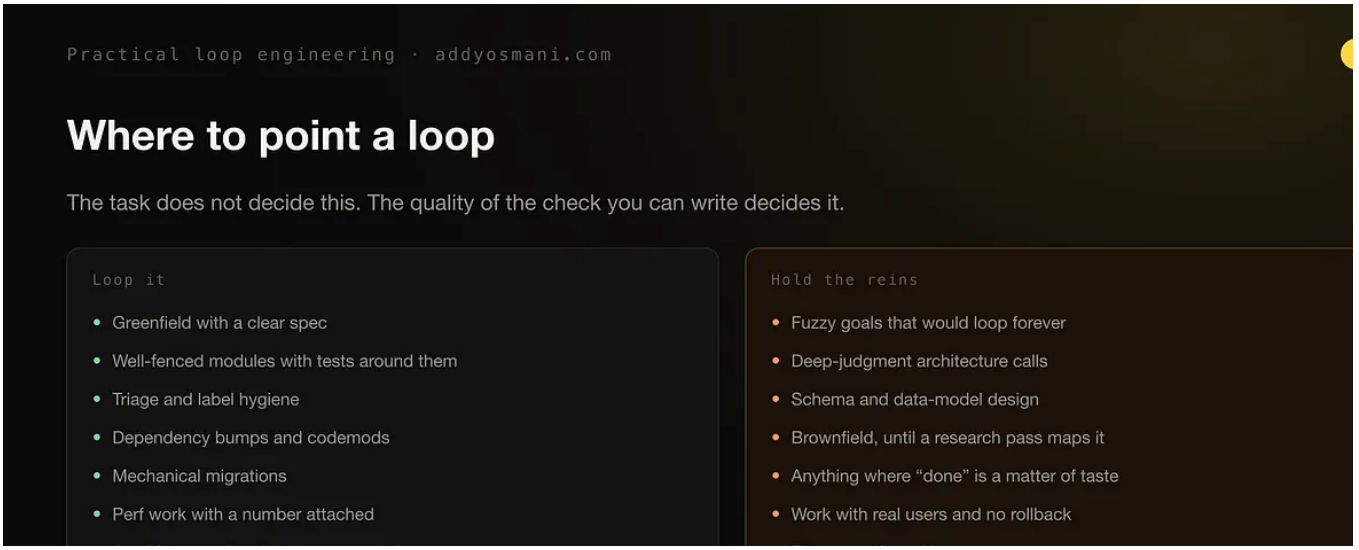
Inside the PR triage, when I have loops and goals working together for me, what up having is a system that allows me to really continue accepting PRs and issues, able to stay on top of what is coming on my plate every day, and especially being to cross-reference. That's been a really big deal for me. If I want to work on a particular goal of, hey, we're going to be redoing this part of the system, and I ne make sure that any issues that happen to touch it are closed as a result of this rev or we're not stepping on anyone else's toes, I can make sure that that's well defin

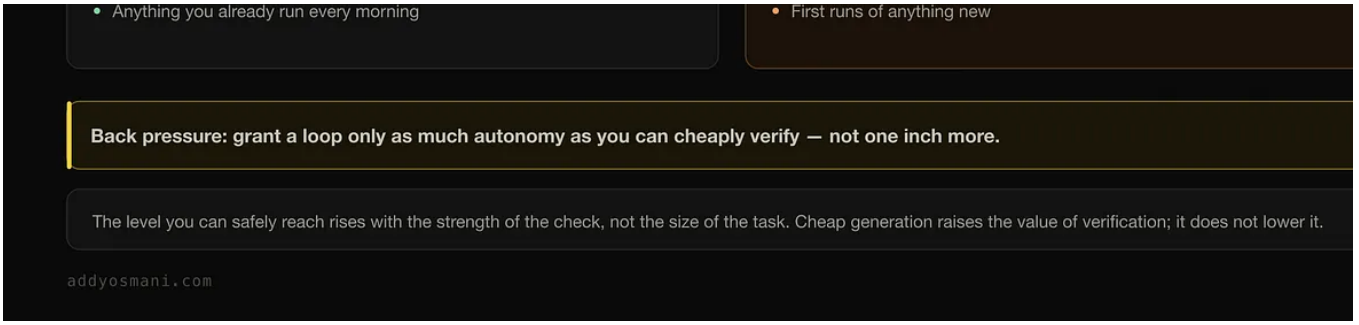
Scheduled tasks are really useful for regularly just reviewing new PRs and closing things that clearly don't fit. One good stopping condition, for example: we have a

contribution guidelines, and our contribution guidelines include things like, hey, currently don't accept translations.



And it's not that we don't care, it's that they're difficult for us to maintain, becau don't speak all the languages often coming in. And so if we tell it, close any issue close any PRs which happen to touch that aspect of our contribution guidelines, something that can do really well when it's on that schedule. And so it reduces th overall batch of things that we need to review.

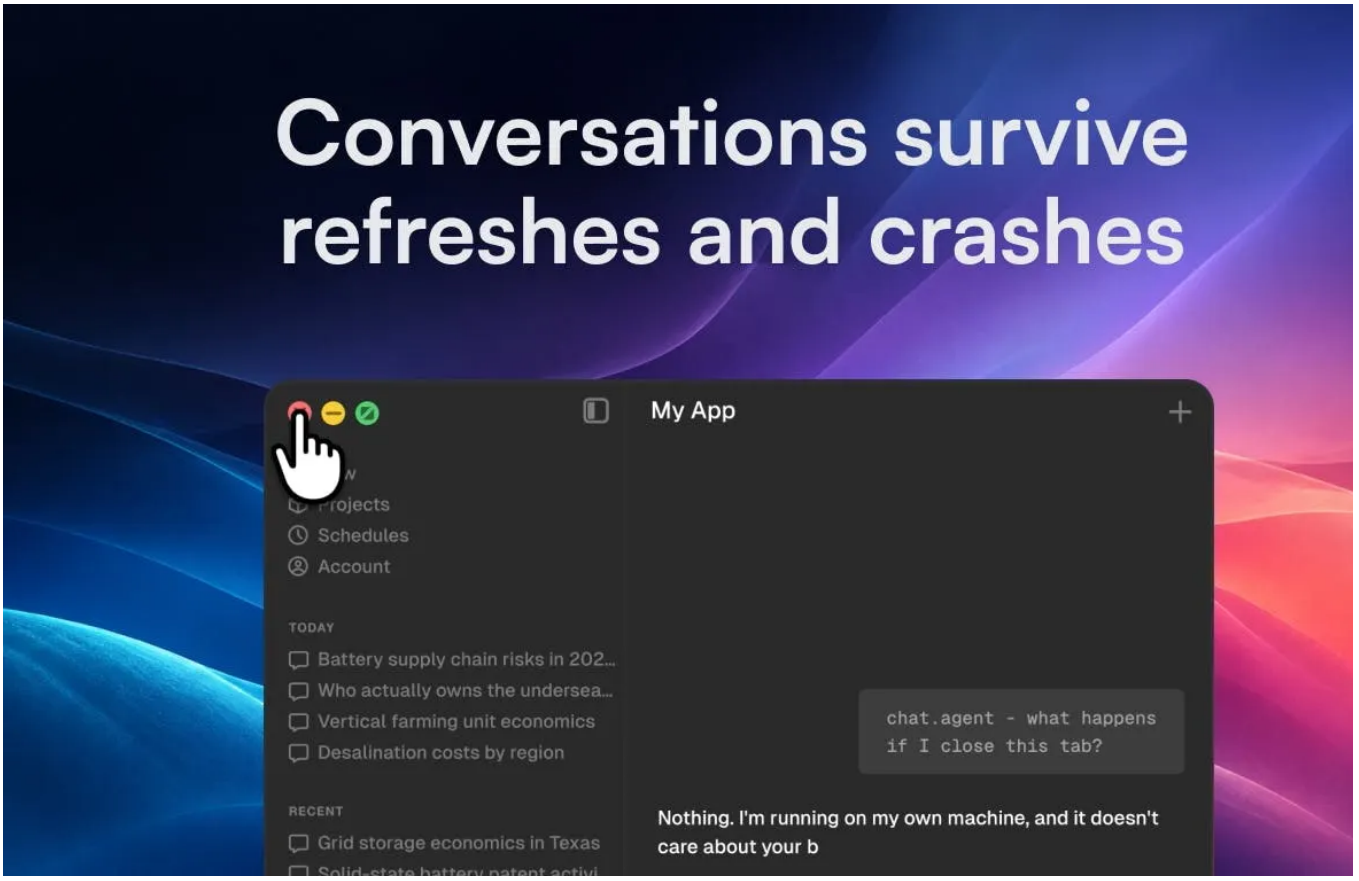




What loops don't buy you

I'm often asked what loop engineering is not a great fit for. Generally speaking if don't have a clear idea of what the end-state/done/good means for your completic may not be the right pattern for your work. For example, a vague goal would be " going until this UI design is good". What does that mean? Good to who? How is being evaluated? Tasks that require human taste, subjective design, or open-ende creative exploration aren't a good fit. When you have a pretty clear idea of the go think loops are a good option to consider.

A final message from our sponsor:



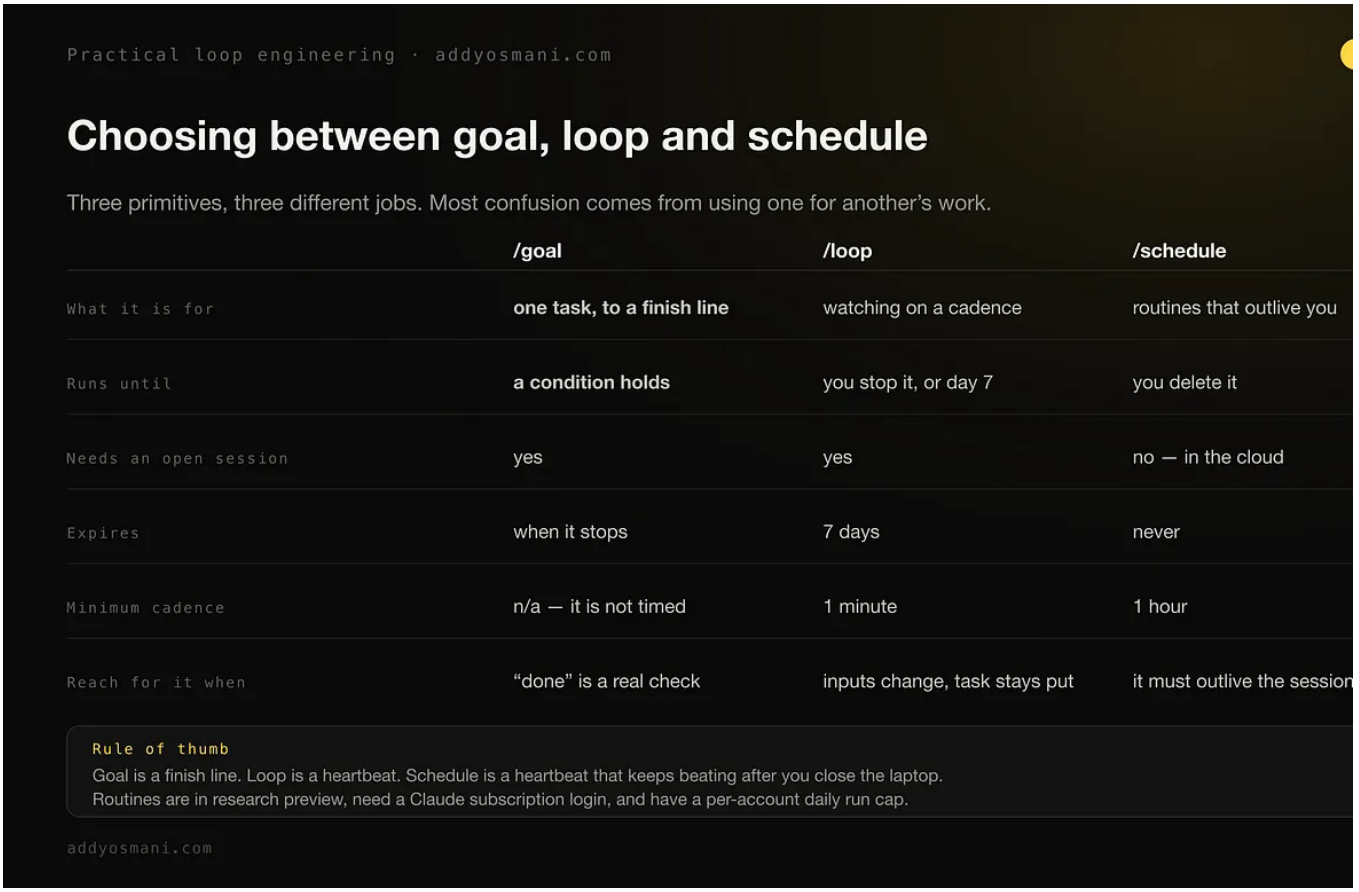
Still hand-rolling durable agents? Trigger.dev's new chat agent (above) makes a multi-turn AI conversation a single durable task - survives redeploys and crashes, pauses for human approval, resumes exactly where it left off. Open source, TypeScript, drops into the Vercel AI SDK. Check them out.

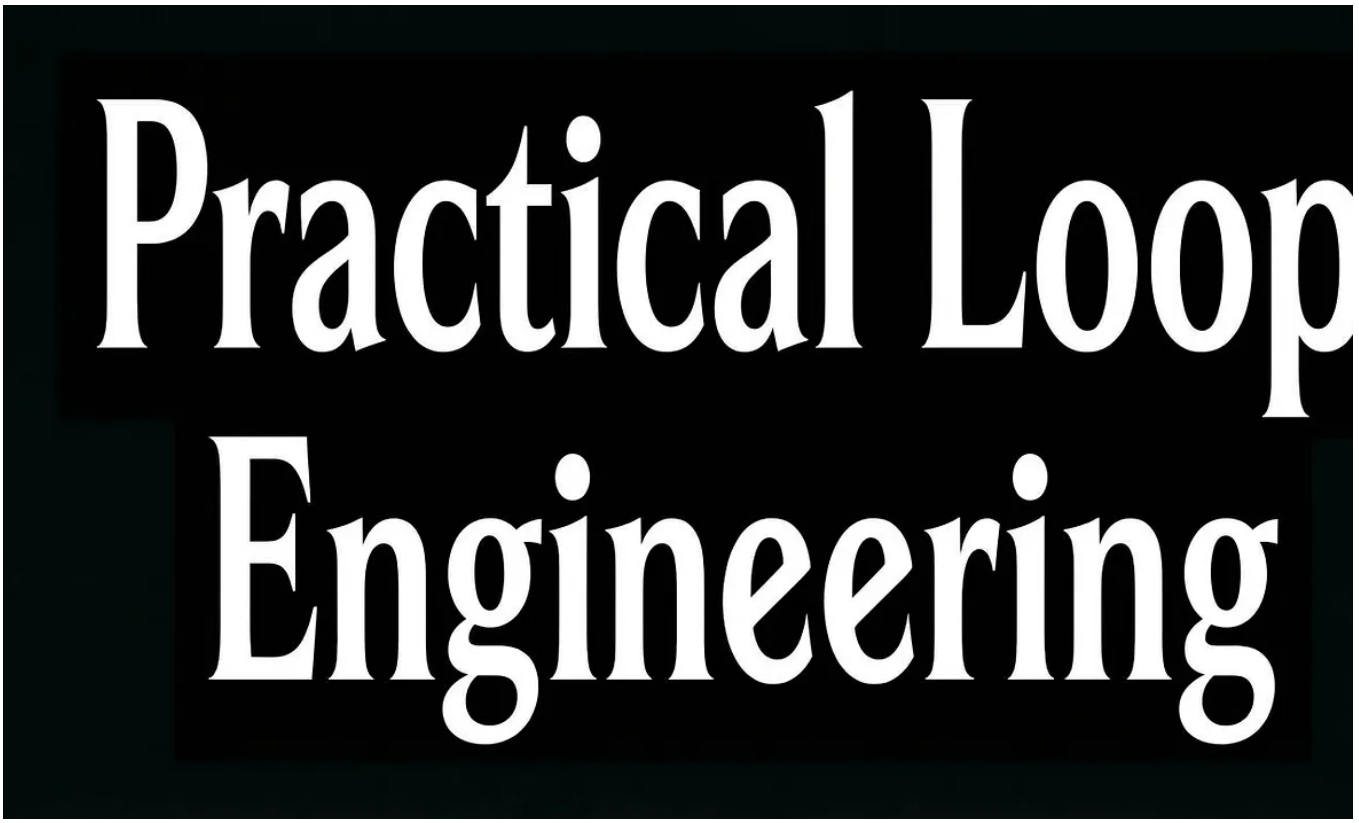
The fine print

One classic sign that you've got a loop spinning in place is the same command be tried over and over without any change in the result. Give the same command a t time with no change from the second and it's probably time to stop.

One bit of fine print worth knowing. Recurring loops expire seven days after crea I'd been telling people this was three days. It's seven. And loops are session-scop they stop when you start a new conversation - though resuming that session with resume or --continue brings back any recurring task still inside its seven-day wir If you need something that outlives your session, /schedule runs it in the cloud.

If there's a check you already run every morning by hand, that's your first loop. M was the pull request pile.





Subscribe to Elevate

By Addy Osmani · Hundreds of paid subscribers

Addy Osmani's newsletter on elevating your effectiveness. Join his community of 600,000 readers social media.

Upgrade to paid

31 Likes · 4 Restacks

Discussion about this post

Comments Restacks

Write a comment...

© 2026 Addy Osmani · Privacy · Terms · Collection notice
Substack is the home for great culture